MODULE 30 · SECTION 4 - PREPARING AND USING DATA FOR ANALYSIS

Preparing Data for ML: BigQuery ML and Features

What the PDE still asks about ML, model types at a glance, feature preprocessing and TRANSFORM, splits and leakage, evaluating and predicting, exporting and registering models.

SECTION 4 of 5 EXAM WEIGHT 15% MODULE 30 of 45

admin@sthithapragna.com

On the PDE, ML is a data job. **Build honest features, and keep them identical in training and serving.**

What the exam still asks about ML

- Since late 2023, ML theory belongs to the **Machine Learning Engineer** exam.
- The PDE keeps **preparing data**: BigQuery ML, features, embeddings and RAG.
- Vertex AI is now **Gemini Enterprise Agent Platform**, since April 2026.
- Option names have not moved: the registry is still **'VERTEX_AI'** in SQL.

> HONEST LIMIT
>
> Older banks ask about dropout and overfitting. Those questions come from the previous exam.

BigQuery ML at a glance

YOU WANT TO	REACH FOR
Predict a **number**	Linear regression, boosted tree
Predict a **category**	Logistic regression, boosted tree, random forest, DNN, AutoML
Find **groups** with no labels	K-means
Recommend products or tariffs	Matrix factorization
Forecast a series	ARIMA_PLUS, or TimesFM with AI.FORECAST
Use a model built **elsewhere**	Import TensorFlow, ONNX, XGBoost, or a remote model

THE PAYOFF

BigQuery ML brings the model to the data. Nothing leaves the warehouse to train.

ARIMA_PLUS or TimesFM?

ARIMA_PLUS

Train your own

- One model per series, trained in SQL
- Holidays, seasonality, tuning
- XREG adds outside drivers, such as weather
- ML.FORECAST to predict

TIMESFM

Pretrained, no training

- A foundation model for time series
- AI.FORECAST, with no model to manage
- Accuracy comparable to ARIMA
- Marked Preview on the overview page

THE TIEBREAKER

No model to train or manage: TimesFM. Need tuning, holidays or extra drivers: ARIMA_PLUS.

Feature preprocessing in BigQuery ML

- BigQuery ML does basic preprocessing **automatically**, such as encoding strings.
- Scale: **ML.STANDARD_SCALER**, ML.MIN_MAX_SCALER. Bucket: **ML.QUANTILE_BUCKETIZE**.
- Encode: **ML.ONE_HOT_ENCODER**, ML.LABEL_ENCODER. Fill gaps: **ML.IMPUTER**.
- Combine: **ML.FEATURE_CROSS**. Inspect: **ML.FEATURE_INFO** shows feature statistics.

FOR LARKSPUR

Bucket tenure, cross tariff with region: two lines of SQL, no separate pipeline.

Preprocess in SELECT, or in TRANSFORM?

IN THE SELECT

Training only

- Applied when the model trains
- You must repeat it at prediction
- Not saved when the model is exported
- Two copies drift apart: skew

IN TRANSFORM

Stored in the model

- Saved inside the model itself
- ML.EVALUATE and ML.PREDICT reapply it
- Exported as a transform SavedModel
- One definition, everywhere

THE SIGNAL

'Predictions drift from evaluation results' after hand-copied preprocessing: move it into TRANSFORM.

Larkspur's churn model, with TRANSFORM

```
1  CREATE OR REPLACE MODEL ml.churn_model
2  TRANSFORM (
3    ML.STANDARD_SCALER(avg_daily_kwh) OVER () AS avg_daily_kwh_scaled,
4    ML.QUANTILE_BUCKETIZE(tenure_months, 5) OVER () AS tenure_bucket,
5    ML.FEATURE_CROSS(STRUCT(tariff_type, region)) AS tariff_region,
6    churned
7  )
8  OPTIONS (model_type = 'BOOSTED_TREE_CLASSIFIER',
9           input_label_cols = ['churned'],
10          model_registry = 'VERTEX_AI')
11 AS SELECT * FROM ml.churn_features WHERE snapshot_date < '2026-07-01';
```

3-5 Scale, bucket and cross, stored inside the model.

10 Registered in the Model Registry, ready to deploy.

REMEMBER

Training stops at July. Later snapshots are kept back to evaluate on unseen months.

Honest evaluation

- **AUTO_SPLIT** is the default. RANDOM, CUSTOM, SEQ and NO_SPLIT are the others.
- Time-ordered data: **SEQ** on a date column, or split by date yourself.
- **Leakage**: a feature known only after the outcome, such as a cancellation request.
- Leaky features give superb evaluation scores, then poor results in production.

> THE TRAP
>
> A random split across years of snapshots lets the model peek at the future.

Scoring the model in SQL

```sql
-- How good is it, on months it has not seen?
SELECT * FROM ML.EVALUATE(MODEL ml.churn_model,
  (SELECT * FROM ml.churn_features WHERE snapshot_date >= '2026-07-01'));

-- Score this month's accounts
SELECT account_id, predicted_churned, predicted_churned_probs
FROM ML.PREDICT(MODEL ml.churn_model,
  (SELECT * FROM ml.churn_features WHERE snapshot_date = '2026-10-01'));
```

2 Precision, recall and ROC AUC for a classifier.

7 The TRANSFORM steps are reapplied automatically.

FOR LARKSPUR

ML.EXPLAIN_PREDICT adds which features drove each prediction. Agents can see why.

Export the model, or register it?

EXPORT MODEL

Files in Cloud Storage

- TensorFlow SavedModel, or XGBoost Booster
- TRANSFORM steps go with it
- SELECT-time features do not
- No export with ARRAY or TIMESTAMP features

MODEL REGISTRY

Deploy for online use

- model_registry = 'VERTEX_AI'
- Versions, evaluations, explanations
- Deploy to an endpoint, no serving container
- Remote models cannot be registered

THE TIEBREAKER

Batch scores in the warehouse: ML.PREDICT. Low-latency scores for an app: register and deploy an endpoint.

Features beyond the warehouse

- A **feature store** serves features that live in BigQuery tables to online models.
- BigQuery stays the source of truth. Online serving reads the latest values fast.
- **Point-in-time** training data uses each feature's value as it was then.
- Dataflow's Enrichment transform can look features up mid-stream, from module 17.

> THE PATTERN
>
> The same feature table feeds training and serving. That is another defence against skew.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Predictions **drift** from evaluation	TRANSFORM clause
Superb scores, **poor in production**	Leakage: drop the feature, split by time
Forecast, **no model to manage**	TimesFM with AI.FORECAST
Forecast with **holidays and drivers**	ARIMA_PLUS or ARIMA_PLUS_XREG
Low-latency scores for an app	Model Registry, deploy to an endpoint
Score **millions of rows** nightly	ML.PREDICT in a scheduled query

THE PATTERN

Honest features, identical everywhere: most ML questions test one of the two.

> Data scientists scale and bucket features in a SELECT before CREATE MODEL. The app team rewrote the same steps in Java for predictions.
>
> Live predictions now differ noticeably from the evaluation results.

What should the team change?

A Move the preprocessing into the model's TRANSFORM clause

B Retrain the model every day, so that it adapts more quickly

C Train on several more years of history, to make it more robust

D Send the app team the features as a nightly CSV extract instead

◆ PAUSE AND DECIDE

What should the team change?

A TRANSFORM

One definition, stored in the model, reapplied everywhere.

B Retrain daily

The two preprocessing copies still differ.

C More history

More data does not fix mismatched features.

D Nightly CSV

Stale, and still a second copy of the logic.

THE SIGNAL

Two copies of preprocessing drifting apart: put it in TRANSFORM.

> A churn model scores an AUC of 0.99 in evaluation, but performs poorly in production.
>
> It trained on three years of monthly snapshots with a random split, and uses a feature called days_since_cancellation_request.

What should the team do first?

A Run more hyperparameter tuning trials on the same training data

B Switch the model type to a DNN, which can learn richer patterns

C Drop features known only after the outcome, and split by time

D Keep AUTO_SPLIT, and simply train the model on more snapshots

◆ PAUSE AND DECIDE

What should the team do first?

A **More tuning**
Tunes harder on a leaky evaluation.

B **A DNN**
A stronger model exploits the leak even better.

C **Fix the leak**
Honest features and a time split give a believable score.

D **More snapshots**
The leak and the random split remain.

THE SIGNAL

Superb evaluation, poor production: leakage. Remove it, and split by time.

> Westgrid wants next week's half-hourly demand forecast for each of 14 grid regions.
>
> Larkspur has no ML engineers, and wants no model to train, tune or manage.

What should the team use?

A. A DNN regressor in BigQuery ML, retrained every week

B. Export the history to Spark, and build a model in MLlib

C. An ARIMA_PLUS model for each region, tuned for holidays

D. TimesFM through AI.FORECAST, over the regional history

◆ PAUSE AND DECIDE

What should the team use?

A DNN

A model to train and manage every week.

B Spark MLlib

Moves the data, and needs ML engineering.

C ARIMA_PLUS

Good forecasts, but models to train and tune.

D TimesFM

Pretrained. One function call, no model to manage.

THE SIGNAL

Forecast with no model to train or manage: TimesFM with AI.FORECAST.

> During a phone call, a call-centre agent's screen must show the customer's churn risk in under 100 milliseconds.
>
> The model was trained in BigQuery ML, and the features are computed at call time.

How should the model be served?

A Run an ML.PREDICT query against BigQuery for every call

B Register the model, then deploy it to an endpoint

C Score every account nightly, and query that table at call time

D Retrain as a logistic regression, so BigQuery answers faster

◆ PAUSE AND DECIDE

How should the model be served?

A A query per call

BigQuery jobs take far longer than 100 ms.

B Registry and endpoint

Online prediction, built for low latency.

C Nightly scores

Features are computed at call time, so last night is stale.

D Simpler model

Still a BigQuery job for every call.

THE SIGNAL

Low-latency predictions for an app: register the model and deploy an endpoint.

Module 30 in five lines

- The PDE tests **data for ML**. Theory moved to the ML Engineer exam in 2023.
- Preprocess in **TRANSFORM**, so training and prediction can never drift.
- Watch for **leakage**, and split time-ordered data by time.
- **TimesFM** for forecasts with nothing to manage, ARIMA_PLUS for tuning.
- **ML.PREDICT** for batch, **registry and endpoint** for low latency.

COMING UP

Next: module 31, embeddings, RAG and LLM-assisted SQL.